

Polymorphism in C# - Comprehensive Study Notes

Overview: What is Polymorphism?

Polymorphism (from Greek: "poly" = many, "morph" = forms) is a fundamental concept in object-oriented programming that allows a single interface to represent multiple concrete implementations. In C#, it enables one type—like a base class—to refer to objects of different derived classes and invoke their behaviors without knowing their specific types.^{[1][2][3]}

Compile-Time Polymorphism (CTP) vs Runtime Polymorphism (RTP)

Compile-Time Polymorphism (Static/Early Binding)

When Decided: At compile time by the compiler

Techniques: Method overloading and operator overloading

Inheritance Required: No

Performance: Faster (no runtime overhead)

Compile-time polymorphism is resolved during compilation. The compiler determines which method to call based on the method signature (name + parameters).^{[2][4][1]}

Method Overloading Example:

```
class Calculator {
    public int Add(int a, int b) {
        return a + b;
    }

    public double Add(double a, double b) {
        return a + b;
    }

    public int Add(int a, int b, int c) {
        return a + b + c;
    }
}

// Usage
```

```
Calculator calc = new Calculator();  
calc.Add(5, 10);           // Calls int version - decided at compile time  
calc.Add(2.5, 3.5);       // Calls double version - decided at compile time
```

Key Points:

- Methods must differ in number, type, or order of parameters^[3]
- Return type alone cannot distinguish overloaded methods
- The compiler selects the appropriate method at compile time^[1]

Runtime Polymorphism (Dynamic/Late Binding)

When Decided: At runtime by the CLR (Common Language Runtime)

Techniques: Method overriding using virtual/override keywords

Inheritance Required: Yes

Performance: Slightly slower (runtime dispatch overhead)

Runtime polymorphism is resolved during execution. The runtime type of the object determines which method executes, not the reference type.^{[4][2]}

Method Overriding Example:

```
class Animal {  
    public virtual void MakeSound() {  
        Console.WriteLine("Animal makes a sound");  
    }  
}  
  
class Dog : Animal {  
    public override void MakeSound() {  
        Console.WriteLine("Dog barks");  
    }  
}  
  
class Cat : Animal {  
    public override void MakeSound() {  
        Console.WriteLine("Cat meows");  
    }  
}
```

```
// Usage
Animal myAnimal = new Dog(); // Base class reference, derived class object
myAnimal.MakeSound();        // Output: "Dog barks" - decided at runtime
```

Comparison: CTP vs RTP

Aspect	Compile-Time	Runtime
Binding Time	Compile time	Runtime
Alternative Names	Static/Early Binding	Dynamic/Late Binding
Techniques	Method/Operator Overloading	Method Overriding
Keywords	None required	virtual, override, abstract
Inheritance	Not required	Required
Performance	Faster	Slightly slower
Resolved By	Compiler	CLR
Based On	Method signature	Object's runtime type

Access Modifiers and Their Behavior During Overriding

Access Modifier Hierarchy (Most to Least Restrictive)

1. **private** - Only accessible within the same class
2. **protected** - Accessible within the same class and derived classes
3. **internal** - Accessible within the same assembly
4. **protected internal** - Accessible within same assembly OR derived classes (union)^{[5][6]}
5. **private protected** - Accessible within same assembly AND derived classes (intersection)^[6]
6. **public** - Accessible from anywhere

Critical Rule: Access Modifiers Must Match

The Golden Rule: When overriding a method in C#, the access modifier in the derived class **MUST BE EXACTLY THE SAME** as the base class method.^{[7][8][9]}

You CANNOT:

- Make an overriding method MORE restrictive
- Make an overriding method LESS restrictive

Reason (Liskov Substitution Principle): If you could change access levels, you'd break the contract established by the base class. A base class reference pointing to a derived object should be able to call all methods available on the base class.^{[10][11]}

Examples:

✗ INVALID - Cannot make MORE restrictive:

```
class Base {
    public virtual void Method() { }
}

class Derived : Base {
    protected override void Method() { } // COMPILE ERROR!
    // Error: Cannot change access modifiers when overriding
}
```

✗ INVALID - Cannot make LESS restrictive:

```
class Base {
    protected virtual void Method() { }
}

class Derived : Base {
    public override void Method() { } // COMPILE ERROR!
    // Error: Cannot change access modifiers when overriding
}
```

☑ VALID - Same access modifier:

```
class Base {
    public virtual void Method() { }
}

class Derived : Base {
```

```
public override void Method() { } // Correct!
}
```

Private Members and Polymorphism

CRITICAL: Private members **CANNOT** participate in polymorphism.^{[12][13]}

Rules for Private:

- Cannot be marked as `virtual`
- Cannot be overridden
- Not inherited by derived classes (though they exist in memory)
- Only visible within the declaring class

```
class Base {
    private virtual void Method() { } // COMPILE ERROR!
    // Error: virtual or abstract members cannot be private
}
```

Why? Polymorphism requires that derived classes can override methods. Private methods are not visible to derived classes, making override impossible.^[13]

Keywords: `virtual`, `override`, `new`, `sealed`

Virtual Keyword

Purpose: Marks a method in the base class as overridable

Where Used: Base class methods, properties, indexers, events^[14]

Rules:

- Cannot be used with `private` or `static`
- Must have an implementation (method body)
- Enables runtime polymorphism

```
class Animal {
    public virtual void MakeSound() {
        Console.WriteLine("Generic animal sound");
    }
}
```

```
    }  
}
```

Important: By default, methods are NON-virtual in C#. Virtual methods allow derived classes to provide new implementations.^{[15][16]}

Override Keyword

Purpose: Provides a new implementation for a base class virtual/abstract method

Where Used: Derived class methods, properties, indexers, events^[17]

Rules:

- Base method MUST be virtual, abstract, or override
- Must have the same signature as the base method
- Access modifier MUST match the base method
- Cannot override non-virtual or sealed methods^{[16][15]}

```
class Dog : Animal {  
    public override void MakeSound() { // Must be public like base  
        Console.WriteLine("Woof!");  
    }  
}
```

Calling Base Implementation:

```
class Dog : Animal {  
    public override void MakeSound() {  
        base.MakeSound(); // Call base class version first  
        Console.WriteLine("Woof!");  
    }  
}
```

New Keyword (Method Hiding)

Purpose: Hides a base class member (NOT true polymorphism)

Difference: This is method HIDING, not overriding^{[18][19][20]}

Key Differences:

Override (Runtime Polymorphism):

```
class Base {  
    public virtual void Test() { Console.WriteLine("Base.Test"); }  
}  
  
class Derived : Base {  
    public override void Test() { Console.WriteLine("Derived.Test"); }  
}  
  
Base obj = new Derived();  
obj.Test(); // Output: "Derived.Test" - Runtime type determines call
```

New (Method Hiding - Compile Time):

```
class Base {  
    public void Test() { Console.WriteLine("Base.Test"); }  
}  
  
class Derived : Base {  
    public new void Test() { Console.WriteLine("Derived.Test"); }  
}  
  
Base obj = new Derived();  
obj.Test(); // Output: "Base.Test" - Reference type determines call  
  
Derived obj2 = new Derived();  
obj2.Test(); // Output: "Derived.Test"
```

Comparison Table:

Feature	override	new
Type of Polymorphism	Runtime (true polymorphism)	Compile-time (hiding)
Base Method Requirement	Must be virtual/abstract	Any method
Method Selection Based On	Object's actual type	Reference variable type

Breaks Inheritance Chain	No	Yes
---------------------------------	----	-----

When to Use new:

- When the base class method is NOT virtual
- When you intentionally want to hide base implementation
- When you don't want polymorphic behavior^{[21][18]}

Sealed Keyword

Purpose: Prevents further overriding in the inheritance chain

Where Used: With override methods in derived classes^{[22][23][24]}

Rules:

- Must be used WITH override
- Prevents further overriding in subclasses
- Used in multi-level inheritance

```
class Animal {
    public virtual void MakeSound() {
        Console.WriteLine("Animal sound");
    }
}

class Dog : Animal {
    public sealed override void MakeSound() { // Sealed override
        Console.WriteLine("Woof!");
    }
}

class Puppy : Dog {
    public override void MakeSound() { } // COMPILE ERROR!
    // Error: cannot override inherited member 'Dog.MakeSound()'
    // because it is sealed
}
```


Use Cases:

- **Security:** Prevent subclasses from changing critical behavior
- **Performance:** Allow compiler optimizations
- **Design:** Stop inheritance chain at a specific point^{[25][22]}

Sealed Classes:

```
sealed class FinalClass { // Cannot be inherited
    public void Method() { }
}

class Derived : FinalClass { } // COMPILE ERROR!
```

Detailed Scenarios and Rules

Multi-Level Inheritance Scenarios

Scenario 1: Virtual → Override → Override

```
class A {
    public virtual void Display() {
        Console.WriteLine("A");
    }
}

class B : A {
    public override void Display() {
        Console.WriteLine("B");
    }
}

class C : B {
    public override void Display() {
        Console.WriteLine("C");
    }
}
```

```
A obj = new C();  
obj.Display(); // Output: "C" - Runtime polymorphism works
```

Scenario 2: Virtual → Override (sealed) → Cannot Override

```
class A {  
    public virtual void Display() {  
        Console.WriteLine("A");  
    }  
}  
  
class B : A {  
    public sealed override void Display() { // Sealed here  
        Console.WriteLine("B");  
    }  
}  
  
class C : B {  
    public override void Display() { } // COMPILE ERROR!  
}
```

Scenario 3: Virtual → new → override Not Possible

```
class A {  
    public virtual void Display() {  
        Console.WriteLine("A");  
    }  
}  
  
class B : A {  
    public new void Display() { // Hiding, not overriding  
        Console.WriteLine("B");  
    }  
}  
  
class C : B {  
    public override void Display() { } // COMPILE ERROR!  
    // Display() in B is not virtual
```

```
}
```

Abstract Methods and Access Modifiers

Rules for Abstract Methods:

- Cannot be private^{[26][27]}
- Can be protected, internal, protected internal, or public
- Derived class override MUST have the same access modifier

```
abstract class Shape {  
    public abstract void Draw();           // ☒ Valid  
    protected abstract void Calculate();  // ☒ Valid  
    private abstract void Invalid();      // X COMPILE ERROR  
}  
  
class Circle : Shape {  
    public override void Draw() { }        // Must be public  
    protected override void Calculate() { } // Must be protected  
}
```

Static Methods and Polymorphism

CRITICAL: Static methods CANNOT participate in polymorphism.^[27]

```
class Base {  
    public static void Method() { }        // Static  
    public virtual static void Test() { }  // X COMPILE ERROR  
}
```

Reason: Static methods belong to the type, not instances. Polymorphism requires instance-level behavior.

Common Pitfalls and Best Practices

Common Mistakes

✗ Mistake 1: Forgetting override keyword

```

class Base {
    public virtual void Method() { }
}

class Derived : Base {
    public void Method() { } // Missing 'override' - This is hiding!
    // Compiler warning: Use 'new' if hiding was intended
}

```

✗ Mistake 2: Trying to override non-virtual method

```

class Base {
    public void Method() { } // Not virtual
}

class Derived : Base {
    public override void Method() { } // COMPILE ERROR!
}

```

✗ Mistake 3: Changing access modifier

```

class Base {
    public virtual void Method() { }
}

class Derived : Base {
    protected override void Method() { } // COMPILE ERROR!
}

```

✗ Mistake 4: Private virtual methods

```

class Base {
    private virtual void Method() { } // COMPILE ERROR!
}

```

Best Practices

☑ **Practice 1:** Always use virtual when you expect overriding

```
class Shape {  
    public virtual double GetArea() { return 0; }  
}
```

☑ **Practice 2:** Use sealed to prevent unwanted overriding

```
class ImportantClass {  
    public sealed override void CriticalMethod() {  
        // Cannot be overridden further  
    }  
}
```

☑ **Practice 3:** Use protected for internal polymorphic methods^[28]

```
class TemplateClass {  
    public void Execute() {  
        Step1();  
        Step2(); // Let derived classes customize  
        Step3();  
    }  
  
    protected virtual void Step2() {  
        // Default implementation  
    }  
}
```

☑ **Practice 4:** Prefer abstract for must-implement methods

```
abstract class Animal {  
    public abstract void MakeSound(); // Forces implementation  
}
```

Summary: Quick Reference

Access Modifier Rules for Overriding

Base Method	Derived Override	Result
public virtual	public override	☑ Valid
protected virtual	protected override	☑ Valid
internal virtual	internal override	☑ Valid
public virtual	protected override	✗ Error
protected virtual	public override	✗ Error
private	Cannot be virtual	✗ Error

Keyword Quick Reference

Keyword	Where Used	Purpose	Can Combine With
<code>virtual</code>	Base class	Allow overriding	access modifiers
<code>override</code>	Derived class	Override virtual/abstract	access modifiers, sealed
<code>new</code>	Derived class	Hide base member	access modifiers, static
<code>sealed</code>	Derived class	Prevent further override	override only
<code>abstract</code>	Base class	Force implementation	access modifiers (not private)

Polymorphism Type Quick Check

Characteristic	Compile-Time	Runtime
Method names	Same	Same
Inheritance	No	Yes
Keywords	None	virtual/override
Decided when	Compile time	Runtime
Based on	Parameters	Object type
Examples	Overloading	Overriding

This comprehensive guide covers all aspects of polymorphism in C# with detailed explanations of compile-time vs runtime polymorphism, access modifier behavior during overriding, and the proper use

of virtual, override, new, and sealed keywords.^{[29][8][23][19][9][2][15][16][21][3][17][22][18][27][1]}

**

1. <https://dotnettutorials.net/lesson/polymorphism-csharp/>
2. <https://www.tutorialspoint.com/difference-between-compile-time-polymorphism-and-runtime-polymorphism>
3. <https://www.geeksforgeeks.org/c-sharp/polymorphism-in-c-sharp/>
4. <https://stackoverflow.com/questions/2152848/compile-time-polymorphism-vs-run-time-polymorphism>
5. <https://www.tutlane.com/tutorial/csharp/csharp-access-modifiers-public-private-protected-internal>
6. <https://ironpdf.com/blog/net-help/csharp-protected/>
7. <https://www.geeksforgeeks.org/java/method-overriding-with-access-modifier/>
8. <https://stackoverflow.com/questions/6236909/why-cant-we-change-access-modifier-while-overriding-methods-in-c>
9. <https://www.scholarhat.com/tutorial/csharp/method-overloading-and-method-overriding-in-csharp>
10. <https://dotnettutorials.net/lesson/liskov-substitution-principle/>
11. <https://blog.ndepend.com/solid-design-the-liskov-substitution-principle/>
12. <https://www.dotnetustad.com/oop-interview/can-private-variables-also-get-inherited>
13. <https://stackoverflow.com/questions/20908078/why-are-private-methods-not-working-with-polymorphism>
14. <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/virtual>
15. <https://www.geeksforgeeks.org/c-sharp/method-overriding-in-c-sharp/>
16. <https://dotnettutorials.net/lesson/function-overriding-csharp/>
17. <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/override>
18. <https://www.geeksforgeeks.org/c-sharp/difference-between-method-overriding-and-method-hiding-in-c-sharp/>
19. <https://www.educative.io/answers/what-is-the-difference-between-the-new-and-override-keywords>
20. <https://dev.to/re nukapatil/method-hiding-and-method-overriding-in-c-14b4>

21. <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/known-when-to-use-override-and-new-keywords>
22. <https://www.programiz.com/csharp-programming/sealed-class>
23. <https://www.c-sharpcorner.com/blogs/sealed-classes-and-methods-in-c-sharp>
24. <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/sealed>
25. <https://ironpdf.com/blog/net-help/csharp-sealed-class-guide/>
26. <https://stackoverflow.com/questions/48295871/what-access-modifier-should-abstract-methods-have-in-an-abstract-class>
27. <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/abstract>
28. <https://www.c-sharpcorner.com/blogs/use-of-protected-methods-in-c-sharp>
29. <https://stackoverflow.com/questions/6162451/the-difference-between-virtual-override-new-and-sealed-override>
30. <https://www.youtube.com/playlist?list=PLib8Q64STW-vAm8qQRHaBOWnNOWR2eznR>
31. <https://www.youtube.com/playlist?list=PL7eie0F75VCznT78pzWvckpShjdcfNLrC>
32. <https://www.youtube.com/playlist?list=PLxOWTEjLkn6rPHhoDfeTT4moLJo1oT9mD>
33. <https://www.youtube.com/playlist?list=PLIAZUEYu2rIP9iDhj665YYcroeTakEtPP>
34. <https://leaders.tec.br/article/d3edc4>
35. <https://www.scholarhat.com/tutorial/csharp/understanding-virtual-override-and-new-keyword-in-csharp>
36. <https://www.programiz.com/csharp-programming/polymorphism>
37. <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/polymorphism>
38. <https://www.geeksforgeeks.org/cpp/compile-time-vs-run-time-polymorphism-difference-in-cpp/>
39. https://www.w3schools.com/cs/cs_access_modifiers.php
40. https://www.w3schools.com/cs/cs_polymorphism.php
41. <https://anilkumarlive.wordpress.com/difference-between-compile-time-and-run-time-polymorphism/>
42. <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/access-modifiers>
43. <https://dotnettutorials.net/course/csharp-dot-net-tutorials/>

44. <https://www.geeksforgeeks.org/c-sharp/csharp-programming-language/>
45. https://www.youtube.com/playlist?list=PL6rpr5gJ1qkEWLOsg_BaFlalzhLCv7BiA
46. https://www.youtube.com/playlist?list=PL99CiVAQoSUS_AbYw2Woe4luLqJ1ecvW0
47. https://www.youtube.com/playlist?list=PLv3i3aAkZiZI_ZvH5Jbi6Byt9EE5XCnqM
48. <https://www.youtube.com/playlist?list=PLu95AwVuIOML6AqeMIGHl2eOLxsXZnd-8>
49. https://www.youtube.com/playlist?list=PL_pbwdlyffsnH3Xlb66FDIHh1yHwWC26I
50. <https://www.youtube.com/playlist?list=PLbAAfbq8xP-JRq3gAV4cxemmGuunWW6o5>
51. https://www.youtube.com/playlist?list=PL5q26DT8Dfjm7Saw0kv_ZBH3DeJM2euWW
52. <https://www.youtube.com/playlist?list=PLdHN14J7CHtZlg2EZKq4Y6lVaALYzGWcu>
53. <https://www.youtube.com/playlist?list=PLj01xxuVwOWrEIpH0kh6Qf8HZ7uKHdz05>
54. <https://www.youtube.com/playlist?list=PL00I4cVdRGwdEWJ9QaDaebekP7wDvbJ2e>
55. https://www.youtube.com/playlist?list=PL3aZbxdSiCbNvcZS8_fcNcMNAxnBkhoa1
56. <https://www.youtube.com/playlist?list=PL3aZbxdSiCbPbxR4RabPqG-gCye3uhkU8>
57. <https://www.youtube.com/playlist?list=PL2UfVCyiPs-3roZUN6P3bQkqU-8HYo5I9>
58. https://www.youtube.com/playlist?list=PLixLDAHnkK8Z9cUW671R_FCqDO_8Wc72D
59. https://www.youtube.com/playlist?list=PLTmuf_R09l2gNLC1QwKrvbdojSiVpVvJ2
60. <https://www.youtube.com/playlist?list=PLcx-MjNucvBrz6PU8LtyE3wKZ09-R5tu>
61. <https://www.youtube.com/playlist?list=PLsQKsFc4KZM2OgBPD5QFkj40It-rSHocn>
62. <https://www.c-sharpcorner.com/article/inheritance-and-polymorphism/>
63. <https://mariusschulz.com/blog/combining-modifiers-in-csharp-protected-internal-and-override-sealed.html>
64. <https://www.infoq.com/articles/overriding-sealed-methods-c-sharp/>
65. <https://www.codu.co/articles/part-7-inheritance-polymorphism-and-interfaces-in-c-7i8gitkl>
66. <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/protected-internal>
67. <https://stackoverflow.com/questions/2375792/overriding-protected-internal-with-protected>

68. <https://stackoverflow.com/questions/13858384/sealed-keyword-in-association-with-override>
69. <https://www.geeksforgeeks.org/c-sharp/sealed-class-in-c-sharp/>
70. <https://www.universalclass.com/articles/computers/object-oriented-concepts-inheritance-and-polymorphism-in-c-programming.htm>
71. <https://github.com/dotnet/csharplang/discussions/5572>
72. <https://ironpdf.com/blog/net-help/csharp-virtual-abstract/>
73. <https://www.sololearn.com/en/Discuss/199131/why-can-t-the-access-level-of-the-overriding-method-be-more-restrictive-than-that-of-the-overridden>
74. https://www.idc-online.com/technical_references/pdfs/information_technology/Understanding_virtual_override_and_new_keyword_in_CSharp.pdf
75. <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/restricting-accessor-accessibility>
76. https://www.w3schools.com/cs/cs_abstract.php
77. <https://stackoverflow.com/questions/3838553/overriding-vs-method-hiding>
78. <https://essentialcsharp.com/access-modifiers>
79. <https://www.c-sharpcorner.com/article/overview-of-abstract-class-and-interface/>
80. <https://www.codingshuttle.com/java-programming-handbook/method-overriding-in-java>
81. <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/inheritance>
82. <https://www.codecademy.com/learn/learn-c-sharp/modules/learn-csharp-inheritance/cheatsheet>
83. <https://dev.to/bytehide/learn-liskov-substitution-principle-in-c-examples-3kla>
84. <https://www.c-sharpcorner.com/blogs/liskov-substitution-principle-in-c-sharp-with-example>
85. <https://essentialcsharp.com/overriding-the-base-class>
86. <https://stackoverflow.com/questions/33115/does-c-sharp-have-the-notion-of-private-and-protected-inheritance>
87. <https://stackoverflow.com/questions/28159268/is-the-liskov-principle-violated-if-overridden-methods-return-a-different-value>
88. https://www.reddit.com/r/csharp/comments/pafr4k/liskov_substitution_just_dont_use_overrides/

89. <https://dotnetsexpert.com/2023/07/30/method-overloading-and-overriding/>

90. https://www.reddit.com/r/csharp/comments/n7bz3w/implementing_only_interface_vs_inheritance_using/